# Fleet Provisioning Subsystem

*SolariNet Monitoring · automated OS install + provisioning + enrollment across distros and architectures · built 2026-06-26*

> **What this is.** A one-click path from the dashboard to a fully installed, configured, and SolariNet-enrolled machine — bare-metal x86 via netboot, Raspberry Pi via native netboot (Pi 5) or a prebuilt image (Pi 4), across **Debian, Ubuntu, and openSUSE** on **x86_64** / **arm64** / **arm32**, all with the **same core software stack**. Ansible + netboot infrastructure lives on `benzene`; the control plane stays in the existing SolariNet server + dashboard on `xenon`.

## Status at build time

| PIECE | STATE |
|---|---|
| benzene infra (Ansible, nginx:8080, TFTP, iPXE, staged proxyDHCP) | installed, non-destructive |
| Dashboard → bridge → script → benzene path | verified end-to-end (staged install + Pi image) |
| Raspberry Pi image build | verified (built + served a 3.8 GB Pi OS arm64 image) |
| Live PXE (netboot actually booting machines) | STAGED — off until the DHCP switch is flipped (see runbook) |
| Server unit tests | 23 / 23 |

## Architecture

The subsystem obeys the project's firm rule — **PHP is read-only; every mutation goes through C via the solariCtl bridge**. Two new operator-gated bridge verbs drive it:

| VERB | FORKS | DOES |
|---|---|---|
| `FLEET_PROVISION` | `deploy/fleet/fleet-provision.sh` | mints the node cert (SIGN), renders the distro's unattended-install config + a per-MAC iPXE entry, pushes them to benzene |
| `FLEET_IMAGE` | `deploy/fleet/fleet-image.sh` | mints the cert, builds a customized Raspberry Pi image on benzene |

Both are detached (double-fork) so the server loop never blocks, and both require an `op=` operator (RBAC). The scripts run on `xenon`, mint certs against the local CA over the bridge (**the CA key never leaves the server**), and SSH to `benzene` to stage artifacts.

```
Dashboard (Provisioning → "Provision a new system")
   └─ POST /api/control/fleet-provision   (PHP: validate + marshal, read-only)
       └─ solariCtl FLEET_PROVISION       (C bridge, operator-gated, detached)
           └─ deploy/fleet/fleet-provision.sh   (on xenon)
               ├─ SIGN CSR → ca.pem/node.pem/node.key   (local CA)
               ├─ render netboot/configs/*.tmpl + ipxe/mac-profile → benzene HTTP root
               └─ stage per-MAC auto-install (inert until PXE is live)
  GET /api/control/fleet-provision?hostname=…  tails run/provision-<host>.log
```

## The shared software stack (identical everywhere)

One logical package set in `deploy/fleet/ansible/group_vars/all.yml` ( `solari_core_packages` ) is applied on every distro and arch; a small `solari_pkg_map` translates the few names that differ between the Debian/Ubuntu and openSUSE families. Core set: sudo curl wget ca-certificates gnupg htop tmux git vim rsync jq unzip lsof chrony python3 net-tools openssh-server + the SolariNet agent. The same list feeds the netboot unattended installs ( `@@PACKAGES@@` ) and the Pi image build, so a machine looks the same however it was provisioned.

## What lives on benzene

| PATH / SERVICE | PURPOSE |
|---|---|
| `/srv/solari-provision/ansible/` | Ansible tree (base-os · common-stack · solarinet-client roles) |
| `/srv/solari-provision/netboot/http/` (nginx :8080) | iPXE scripts, rendered installer configs, enrollment material, images |
| `/srv/solari-provision/netboot/tftp/` | iPXE binaries (x86_64 UEFI, arm64 UEFI, pxelinux) |
| `/srv/solari-provision/pi/` | Pi image builder + Pi 5 netboot scaffolding |
| `solari-netboot.service` + `/etc/solari-provision/dnsmasq-proxy.conf` | **staged, disabled** proxyDHCP+TFTP responder (isolated from libvirt's dnsmasq) |
| `/srv/solari-provision/staging/go-live.sh` | one-command flip to activate/deactivate live PXE |

> benzene's existing roles (MariaDB, Redis, Samba, libvirt VMs, desktop) were **left untouched** — the provisioning stack runs alongside them, and its PXE responder is a *separate* dnsmasq bound to `eno1`, never the libvirt one.

## Go-live runbook (Saturday) — activating live netboot

Everything above is staged and inert. Bare-metal machines only netboot once DHCP tells them where the boot server is. Pick **one** of these; both coexist with the existing gateway DHCP.

### Option A — proxyDHCP on benzene (self-contained, reversible)

A dedicated dnsmasq in *proxy* mode answers only PXE boot requests; it never hands out IP leases, so it does not conflict with the gateway's DHCP.

```
# first, fetch the netboot installer kernels (one-time, ~minutes):
deploy/fleet/fetch-images.sh --distro all --arch x86_64

# then flip PXE live on benzene:
```

```
/srv/solari-provision/staging/go-live.sh          # enable + open firewall
/srv/solari-provision/staging/go-live.sh --off    # fully reverse it
```

### Option B — UniFi DHCP options (no responder on benzene)

Add these to the DHCP scope in the UniFi controller (Network → the LAN → DHCP), then leave benzene's responder disabled:

```
Network boot / PXE:
  Enabled:      yes
  Server (next-server):  10.5.2.50        # benzene
  Filename (BIOS):       pxelinux.0
  Filename (UEFI x64):   ipxe-x86_64.efi
  Filename (UEFI arm64): snp-arm64.efi
# iPXE then chains to  http://10.5.2.50:8080/ipxe/boot.ipxe
```

> **Network-safety note.** Do not run a second full DHCP *server* on the flat 10.0.0.0/8 — only proxyDHCP (Option A) or the existing UniFi scope (Option B). The user asked to keep the live network undisturbed until Saturday; nothing in tonight's deploy touches DHCP.

## Provisioning flows

### Bare-metal x86 (Debian / Ubuntu / openSUSE)

- Dashboard → Provisioning → **Provision a new system** → "OS install", pick distro/arch/role, enter the target's **MAC** and a hostname → **Stage install**.
- The machine auto-installs unattended on its next network boot (per-MAC iPXE profile), runs the common package stack, and enrolls into SolariNet. An un-staged machine is never touched (the per-MAC chain 404s and it boots local disk).

### Raspberry Pi

- **Pi 5** — native netboot (TFTP/HTTP root served from benzene); scaffold with `deploy/fleet/pi/pi5-netboot/`. Gated behind the same PXE switch.

- **Pi 4** — dashboard "Pi image" mode builds a customized Pi OS image (hostname, SSH key, core stack, SolariNet firstboot enrollment). Download from `http://benzene:8080/images/<host>-<arch>.img` and flash to SD/USB. Fully working today, no PXE needed.

## Security model

- Both verbs are operator-gated (RBAC); PHP validates every field and holds no CA material.
- Node certs are minted on xenon via the bridge `SIGN` — the CA private key never leaves the server process, never touches benzene or the deploy scripts.
- Enrollment material is staged under an **unguessable per-host token dir** ( `…/enroll/<random>/` ) and is not directory-listed; it should be removed after the node's first boot. Follow-up: replace with a one-time pull token for defence in depth.
- Installed hosts get key-only SSH (password auth disabled, admin account with a locked password + the operator's public key).

## Outstanding / follow-ups

- **Flip PXE live** Saturday via the runbook, then boot a real target to validate an unattended install end-to-end (only the final bare-metal wipe step is untested — it needs real hardware set to netboot).
- **fetch-images.sh** URLs (esp. Ubuntu 24.04 netboot) may need adjusting as releases move.
- arm64 *non-Pi* netboot installers, and arm32/mips static agent builds, are future work (see the heterogeneous-client roadmap).
- Enrollment auto-delivery hardening (one-time token) as noted above.

---

SolariNet Fleet Provisioning — generated 2026-06-26. Infrastructure staged non-destructively; live PXE awaits the Saturday go-live. Ansible + dashboard + Pi imaging verified end-to-end.